



MCP + Playwright + Jira: How I Automated My Entire QA Workflow End-to-End



Mykola Nesvitii

Follow

8 min read · Apr 23, 2026



80



3



A technical deep-dive into running Claude API as a headless QA agent — from Jira ticket to merged PR, without touching the keyboard.

...

Stack: Claude API · MCP (Playwright + Jira) · TypeScript · GitHub Actions

...

The Problem Nobody Talks About

Everyone talks about how AI can write Playwright tests. What they don't talk about is the **scaffolding tax** — the work that happens before and after the test file itself.

Before a test: someone has to read the Jira ticket, understand the acceptance criteria, map it to the existing page object model, identify which fixtures to reuse, and figure out which test suite it belongs in. After a test: someone has to update the Jira ticket, link the PR, add the test case to the test management system, and close the loop.

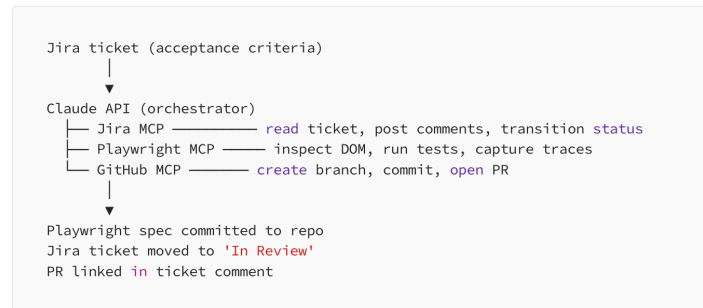
That scaffolding was costing my team more time than writing the actual test. And when I started using Claude API directly — not as a chat assistant, but as a structured agent with tool access — I realized all of it was automatable.

a structured agent with tool access — I realised all of it was automatable.

The test file is 20% of the work. The other 80% is reading tickets, wiring context, and updating status. That 80% is where an agent earns its keep.

The Architecture

The full pipeline looks like this:



No human touches a keyboard between “assign ticket” and “review PR”. That’s the goal, and that’s what’s running in production.

Why Claude API directly, not Cursor or Cline?

I tried both. The issue with IDE-based agents is that they’re optimised for interactive, human-in-the-loop sessions. My workflow needed to run **headlessly** — triggered by a webhook, completing without supervision, and producing a deterministic output.

Using the Claude API directly gave me full control over the system prompt, the tool definitions, the retry logic, and the output format. It also meant I could embed the agent inside our existing CI infrastructure rather than running it on someone’s laptop.

The Hard Part: Making the Agent Respect Conventions

This is where most blog posts gloss over the real pain. Getting Claude to write a Playwright test is trivial. Getting it to write a test that matches your project’s conventions — your page object model, your fixture patterns, your naming scheme, your assertion style — is the **actual engineering problem**.

Here is what didn’t work, and what finally did.

Attempt 1: Describing conventions in the system prompt

My first approach was to write a detailed system prompt explaining the conventions: *“Use the BasePage class, fixtures go in fixtures/, use data-testid selectors, etc.”*

The agent followed about 70% of them. The other 30% it either ignored or hallucinated its own patterns. In a 200-line test file, that’s a lot of review notes.

Attempt 2: Attaching example files

Better. I included 2–3 existing test files as few-shot examples in the user message alongside the Jira ticket. The agent’s conformance jumped to roughly 85%. But it still invented selectors that didn’t exist in the DOM and occasionally bootstrapped its own helper functions instead of using existing ones.

Attempt 3: AGENTS.md + live DOM inspection via Playwright

MCP

This is what works in production. Two changes made the difference.

First, an **AGENTS.md** file that is **strict, not descriptive**. Not “*prefer data-testid selectors*” but “*ONLY use data-testid selectors. Any test using CSS class selectors will be rejected in review. Any test using XPath will be deleted without comment.*”

Second, the **Playwright MCP** tool is called **before writing a single line**. The agent navigates to the page under test, inspects the DOM, and extracts the actual `data-testid` attributes that exist. It writes selectors from evidence, not from memory.

Here's an excerpt from our **AGENTS.md** :

```
## SELECTORS
- ONLY use data-testid attributes: page.getByTestId('submit-btn')
- CSS class selectors: PROHIBITED
- XPath: PROHIBITED
- Text-based selectors: allowed only as fallback for third-party components

## PAGE OBJECTS
- All page interactions must go through a Page Object in /pages/
- Never call page.click() directly in a spec file
- Page Object constructor signature: constructor(readonly page: Page) {}

## FIXTURES
- Extend the base fixture in /fixtures/base.ts
- Never use test.beforeEach() for auth – use the authed fixture
- Data setup goes in /fixtures/data-factory.ts
```

The **AGENTS.md** is injected into the system prompt verbatim. It's maintained by the team, version-controlled alongside the tests, and updated whenever a new convention is adopted. When the agent violates a rule, we don't fix the agent — we make the rule in **AGENTS.md** more explicit.

Don't chase the agent's mistakes with prompt tweaks. Make the contract (AGENTS.md) more precise. The agent will follow a clear, unambiguous rule far more reliably than a nuanced description.

The Jira Integration in Detail

The Jira MCP does more than just read ticket descriptions. Here is the full sequence of operations the agent performs on a single ticket:

1. **Fetch ticket fields** — summary, description, acceptance criteria, linked tickets, labels, and story points.
2. **Post a comment** — “*Starting automated test generation. Will update when PR is ready.*”
3. **Transition status** — “To Do” → “In Progress”
4. **After PR is created** — post the PR link as a comment.
5. **Transition status** — “In Progress” → “In Review”
6. **If tests fail in CI** — post failure summary, transition back to “In Progress”

Steps 4–6 are what close the feedback loop. The ticket tells the story of what happened — not just “*tests written*” but a traceable audit trail from ticket to PR to CI result.

Extracting acceptance criteria reliably

Jira ticket descriptions are freeform. Some engineers write Gherkin (Given/When/Then). Some write bullet lists. Some write prose. The agent

needs to extract testable assertions from all of these formats.

Get Mykola Nesvitii's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☒ Remember me for faster sign in

I solved this with a **normalisation step** — a dedicated Claude API call before the main agent loop, whose only job is to convert the raw ticket description into a structured JSON object:

```
const normalize = await claude.messages.create({
  model: 'claude-sonnet-4-20250514',
  max_tokens: 1000,
  system: `You extract testable assertions from Jira tickets.
  Return ONLY valid JSON. No preamble. No markdown.`,
  messages: [{
    role: 'user',
    content: `Ticket description:\n${ticket.description}\n\n
    Return JSON: { assertions: string[], preconditions: string[],
    userRole: string, feature: string }`
  }]
});

const { assertions, preconditions, userRole, feature } =
  JSON.parse(normalize.content[0].text);
```

This structured output feeds into the main agent as clean, unambiguous test requirements. The agent never has to guess what the ticket is asking for.

Playwright MCP: More Than a Code Writer

The Playwright MCP tool serves two distinct roles in this workflow, and the second one is more important than the first.

Role 1: DOM inspection before writing

Before generating any test code, the agent uses the Playwright MCP to open the feature under test locally (or in a staging environment), navigate to the relevant page, and extract all `data-testid` attributes in the relevant UI region.

This gives the agent a **ground-truth selector map**. It isn't guessing — it knows which elements actually exist and what they're called.

Role 2: Debugging on failure

This is the part that changed how I think about AI-assisted testing. When a generated test fails on first run, instead of the agent hallucinating a fix from the error message alone, it uses the Playwright MCP to:

- Navigate to the failing page with a headed browser
- Execute the failing steps up to the point of failure
- Take a screenshot and inspect the DOM state at the moment of failure
- Compare the expected selector against what actually exists in the DOM

The fix that comes out of this process is almost always correct on the first attempt, because it's based on **observation, not inference**.

```
let attempts = 0;
while (attempts < 3) {
  const result = await runPlaywrightTest(specPath);
```

```

    if (result.passed) break;

    // Agent inspects the failure before fixing
    await agent.call('playwright_navigate', { url: result.failingUrl });
    await agent.call('playwright_screenshot', {});
    await agent.call('playwright_get_dom', { selector: result.failingSelector });

    const fix = await claude.messages.create({
      messages: [{ role: 'user', content:
        `Test failed at: ${result.error}\n
        Current DOM state: ${domSnapshot}\n
        Screenshot attached.\n
        Fix the spec. Follow AGENTS.md conventions.` }],
    });

    applyFix(fix);
    attempts++;
  }
}

```

What It Looks Like Day to Day

In practice, the workflow is triggered by a Jira automation rule: when a ticket with the label `needs-e2e` is moved to "In Progress" and assigned to the QA bot user, a webhook fires.

From that point, my job is to wait for the PR notification in Slack, review the diff, run the tests locally if anything looks off, and approve or request changes. The review takes **10–15 minutes** on a typical ticket.

What I am *not* doing: reading the ticket, setting up test data, writing the spec, fixing selector issues, updating Jira, or opening the PR. The agent handles all of that.

Time comparison — single feature ticket Before: ~3.5 hours (ticket read, test design, writing, debugging, Jira update, PR). After: ~15 minutes of review.
Agent runtime: 8–12 minutes.

What It Doesn't Do Well (Yet)

Complex multi-actor flows. Tests that require two users interacting simultaneously — for example, an admin approves and the user sees the update in real time — require significant hand-holding. The fixture setup for multi-session state is usually wrong on the first attempt.

Tickets with ambiguous acceptance criteria. If the ticket says “*verify the dashboard loads correctly*”, the agent produces a weak test that checks for the page title and calls it done. Garbage in, garbage out.

Visual regression. Any test that requires pixel-level or layout comparison is out of scope. The agent has no concept of “*this looks wrong*”.

Third-party components without data-testid. When a UI relies on a library component that doesn't expose `testid` attributes, the fallback selector logic is brittle and often wrong.

How to Get Started

If you want to build something similar, here is the order I'd recommend:

1. **Start with the AGENTS.md.** Write your conventions down before you write a single line of agent code. This is the most valuable artifact in the entire system.
2. **Wire up Jira MCP read-only first.** Just have the agent read a ticket and print the normalised JSON. Validate the extraction quality before going further.
3. **Add Playwright MCP for DOM inspection only** — no test writing yet.
Verify the agent can reliably extract selectors from your staging

...only the agent can reliably extract selectors from your staging environment.

- 4. **Generate your first test with a human in the loop.** Review the output and update AGENTS.md based on what it gets wrong.
- 5. **Automate the Jira write-back** once test quality is where you want it.
- 6. **Add the debug loop last**, once you have confidence in the baseline.

The full pipeline took about three weeks of part-time work to get to a state I trusted in production. The AGENTS.md has been updated probably 40 times since then.

Final Thought

The agent doesn't replace the QA engineer — it replaces the grind. What's left is the part that actually requires judgement: reviewing whether the test captures the right intent, catching edge cases the ticket didn't mention, and deciding when a failure means “fix the test” versus “fix the product”.

That's a better use of a QA engineer's time. The agent handles the pipeline. You handle the thinking.

Give your agent a strict written contract (AGENTS.md), ground truth from the DOM (Playwright MCP), and structured input from your ticket system (Jira MCP). Everything else is just plumbing.



80



3



Written by Mykola Nesvitii

84 followers · 1 following

Follow

Responses (3)



Write a response

What are your thoughts?



Roman Marinsky

May 3

...

Bla bla bla



5



1 reply

Reply



Cameron Osborne

May 21

...

Great post. The key insight is that the hard part is not “Claude writes a Playwright test,” but the scaffolding around it: ticket interpretation, convention retrieval, DOM grounding, test execution, repair, PR creation, and Jira closure.

One thing... [more](#)



2

Reply



Hien Luong

6 days ago

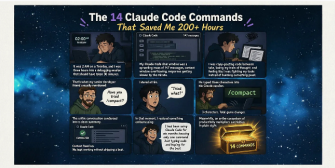
...

Great post. Love the breakdown. As an engineer, when finishing a feature, I usually have multiple test scenarios, and have to pick what test case should get converted to e2e. The hard part is to write down the test scenario. Open question that I... [more](#)



Reply

Recommended from Medium



In Towards AI by Mouez Yazidi · Apr 26

I Wasted 6 Months Using Claude Code Wrong. Here Are the 14...
From frustrated beginner to power user: The hidden command ecosystem nobody talks...

1.6K 36 45



In UX Planet by Nick Babich · Apr 6

MCP is Dead
Why you should avoid using MCP in Claude Code and what to use instead

3.5K 203 88



In Artificial Corner by The PyCoach · Apr 29

I Tried 100 Claude Skills. These Are The Best
Tested, ranked, and ready to use

3.7K 71 74



In Data Science Collective by Anubhav · Apr 24

I Spent 6 Months Tuning Claude Code. Here's the Exact Setup Tha...
CLAUDE.md, subagents, hooks, skills, worktrees, and the five MCP servers that ear...

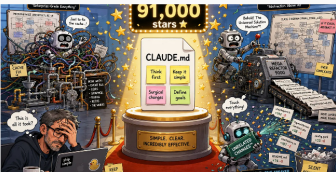
2.1K 21 27



In Write A Catalyst by Suraj Jha · May 12

I Deleted Notion and Obsidian. Here's What Replaced Them — a...
Two of the most popular productivity apps in the world. Both are gone in a week. And...

2.1K 81 55



In Towards Deep Learn... by Sumit Pand... · May 8

A Single CLAUDE.md File Went Viral. The Reason Is...
91,000 stars on GitHub. No code. Four rules from Andrej Karpathy that every coding...

2.7K 37 48

See more recommendations